📄 ✅ WEEK 1 REPORT

Week 1: Threat Intelligence Feed Integration

Objective

The objective of Week 1 was to set up the development environment and implement automated threat intelligence collection from multiple OSINT (Open Source Intelligence) sources.

---

1. Environment Setup

The project environment was configured on a Windows system.

Tools Installed:

- Python 3.x
- MongoDB (local database)
- Git (version control)
- VS Code (code editor)

Python Libraries Installed:

pip install requests pymongo python-dotenv

---

2. Project Structure

The project was organized into modular components:

threat-intelligence-platform/

```
│
├── src/
│   ├── collectors/
│   │   ├── __init__.py
│   │   ├── base_collector.py
│   │   ├── alienvault_collector.py
│   │   ├── abuseipdb_collector.py
```

```
│   │   ├── virustotal_collector.py
│   │   ├── phishtank_collector.py
│   │   ├── threatfox_collector.py
│   │   ├── urlhaus_collector.py
│   │   └── feodo_collector.py
│   │
│   ├── database/
│   │   ├── __init__.py
│   │   └── mongodb_connector.py
│   │
│   ├── processors/
│   │   ├── __init__.py
│   │   ├── normalizer.py
│   │   └── deduplicator.py
│
├── docs/
│   └── week1_report.pdf
│
├── assets/
│   └── screenshots.png
│
├── run_collectors.py
├── .env
├── requirements.txt
├── README.md
└── .gitignore
```

---

3. MongoDB Integration

MongoDB was used to store collected threat indicators.

Database Details:

- Database Name: threat_db
- Collection Name: threats

Features:

- Stores threat indicators (IP, domain, hash)
- Supports fast querying
- Avoids duplicate data using update_one()

---

4. OSINT API Integration

Threat intelligence was collected from multiple sources:

Integrated APIs:

- AlienVault OTX
- VirusTotal
- AbuseIPDB

Data Collected:

- Malicious IP addresses
- Domains
- URLs
- File hashes

---

5. Collector Implementation

Each OSINT source was implemented as a separate Python collector.

Example Flow:

1. Connect to API
2. Fetch data
3. Extract indicators
4. Format data

5. Send to database

---

6. Data Storage Logic

Threat indicators were stored using MongoDB with deduplication.

Key Logic:

```
collection.update_one(
  {"indicator": ind["indicator"], "type": ind["type"]},
  {"$set": ind},
  upsert=True
)
```

Benefit:

- Prevents duplicate entries
- Updates existing records

---

7. Execution Script

A central script was created:

run_collectors.py

- Runs all collectors
- Collects data from APIs
- Stores data in MongoDB
- Runs in loop (every 5 minutes)

---

8. Output

Sample output after running collectors:

- AlienVault: 132 indicators
- VirusTotal: 1 indicator
- AbuseIPDB: 0 indicators
- Total Collected: 133 indicators

All data successfully stored in MongoDB.

9. Challenges Faced

Issue	Solution
API Errors (401/403)	Fixed API keys in .env
Duplicate Data	Used MongoDB update_one()
Infinite Loop	Controlled execution
Database Errors	Corrected DB name and collection

---

10. Learning Outcomes

- Understanding of OSINT threat intelligence
- API integration using Python
- MongoDB database handling
- Data collection automation
- Basic cybersecurity concepts

---

11. Conclusion

Week 1 successfully implemented a working threat intelligence collection system. The platform can now automatically fetch and store threat indicators from multiple OSINT sources, forming the foundation for further development.

⚙️ ✅ STEP-BY-STEP RUNNING (WINDOWS)

🟢 1. Open Project Folder

cd C:\threat-intelligence-platform

🟢 2. Install Requirements

pip install -r requirements.txt

🟢 3. Setup .env file

Create .env in root folder:

MONGO_URI=mongodb://localhost:27017/
DB_NAME=threat_db

ALIENVAULT_API_KEY=your_key
VIRUSTOTAL_API_KEY=your_key
ABUSEIPDB_API_KEY=your_key

🟢 4. Start MongoDB

👉 Make sure MongoDB is running:

mongod

OR (if already installed as service, skip)

🟢 5. Run Project

python run_collectors.py

🟢 6. Expected Output

🚀 Starting collection cycle...
AlienVault: 132
VirusTotal: 1
AbuseIPDB: 0
Collected: 133

After cleaning: 133

✅ Stored in MongoDB

🟢 7. Check Data in MongoDB

Open Mongo shell:

```
mongosh
```

Then:

```
use threat_db
db.threats.find().pretty()
```